In [2]:

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

In [4]:

```python
# ==============================================================
# 1. SELECT KBEST FEATURE SELECTION
# ==============================================================
def selectkbest(indep_X, dep_Y, n):
    test = SelectKBest(score_func=chi2, k=n)
    fit1 = test.fit(indep_X, dep_Y)
    selectk_features = fit1.transform(indep_X)
    return selectk_features
```

In [12]:

```python
# ==============================================================
# 2. TRAIN TEST SPLIT + STANDARD SCALING
# ==============================================================

def split_scalar(indep_x, dep_y):

    # Split the data into training and testing data
    x_train, x_test, y_train, y_test = train_test_split(
        indep_x,
        dep_y,
        test_size=0.25,
        random_state=0,
        stratify=dep_y
    )

    # Create StandardScaler object
    sc = StandardScaler()

    # Fit scaler only on training data
    x_train = sc.fit_transform(x_train)

    # Apply the same scaling to test data
    x_test = sc.transform(x_test)

    # Return the four datasets
    return x_train, x_test, y_train, y_test
```

In [14]:

```python
# ==============================================================
# 3. ACCURACY PREDICTION
# ==============================================================
def accuracy_prediction(classifier, X_test, y_test):
    y_pred = classifier.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    return accuracy
```

In [15]:

```python
# ==============================================================
# 4. LOGISTIC REGRESSION
# ==============================================================
def Logistic(X_train, y_train, X_test, y_test):
    classifier = LogisticRegression(max_iter=1000)
    classifier.fit(X_train, y_train)
    accuracy = accuracy_prediction(
       classifier,
       X_test,
       y_test
   )
    return accuracy
```

In [16]:

```python
# ==============================================================
# 5. SVM - LINEAR
# ==============================================================
def svm_linear(X_train, y_train, X_test, y_test):
    classifier = SVC(kernel='linear')
    classifier.fit(X_train, y_train)
    accuracy = accuracy_prediction(
    classifier,
    X_test,
    y_test
 )
    return accuracy
```

In [19]:

```python
# ==============================================================
# 6. SVM - NON LINEAR / RBF
# ==============================================================
def svm_NL(X_train, y_train, X_test, y_test):
    classifier = SVC(kernel='rbf')
    classifier.fit(X_train, y_train)
    accuracy = accuracy_prediction(
    classifier,
    X_test,
    y_test
  )
    return accuracy
```

In [20]:

```python
# ==============================================================
# 7. DECISION TREE
# ==============================================================
def Decision(X_train, y_train, X_test, y_test):
    classifier = DecisionTreeClassifier(
    random_state=0
    )
    classifier.fit(X_train, y_train)
    accuracy = accuracy_prediction(
    classifier,
    X_test,
    y_test
   )
    return accuracy
```

In [22]:

```python
# ==============================================================
# 8. RANDOM FOREST
# ==============================================================
def random(X_train, y_train, X_test, y_test):
    classifier = RandomForestClassifier(
    n_estimators=10,
    random_state=0
  )
    classifier.fit(X_train, y_train)
    accuracy = accuracy_prediction(
    classifier,
    X_test,
    y_test
 )
    return accuracy
```

In [40]:

```python
# ==============================================================
# 9. CREATE RESULT DATAFRAME
# ==============================================================
def selectk_classification(
    acclog,
    accsvml,
    accsvmnl,
    accdes,
    accrf
):
    dataframe = pd.DataFrame(
    index=['ChiSquare'],
    columns=[
        'Logistic',
        'SVMl',
        'SVMnl',
        'Decision',
        'Random'
    ]
)
    for number, idex in enumerate(dataframe.index):
        dataframe.loc[idex, 'Logistic'] = acclog[number]
        dataframe.loc[idex, 'SVMl'] = accsvml[number]
        dataframe.loc[idex, 'SVMnl'] = accsvmnl[number]
        dataframe.loc[idex, 'Decision'] = accdes[number]
        dataframe.loc[idex, 'Random'] = accrf[number]
        return dataframe
```

In [43]:

```python
# ==============================================================
# 10. LOAD DATASET
# ==============================================================
dataset1 = pd.read_csv(
    "Social_Network_Ads.csv",
    index_col=None
)
# ==============================================================
# 11. CREATE DUMMY VARIABLES
# ==============================================================
df2 = dataset1
df2 = pd.get_dummies(
    df2,
    drop_first=True
)
# ==============================================================
# 12. SEPARATE X AND Y
# ==============================================================
indep_X = df2.drop(
    'Purchased',
    axis=1
)
dep_Y = df2['Purchased']
```

In [44]:

```python
# ==============================================================
# 13. SELECT TOP 5 FEATURES
# ==============================================================
kbest = selectkbest(
indep_X,
dep_Y,
5
)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\feature_selection\_univariate_selection.py:782: User
Warning: k=5 is greater than n_features=4. All the features will be returned.
  warnings.warn(
```

In [45]:

```python
# ==============================================================
# 14. CREATE EMPTY LISTS
# ==============================================================
acclog = []
accsvml = []
accsvmnl = []
accdes = []
accrf = []
```

In [46]:

```python
# ==============================================================
# 15. TRAIN TEST SPLIT
# ==============================================================
X_train, X_test, y_train, y_test = split_scalar(
kbest,
dep_Y
)
```

In [47]:

```python
# ==============================================================
# 16. RUN MACHINE LEARNING MODELS
# ==============================================================
# kbest is a NumPy array.
# kbest.shape[1] gives the number of selected features.

for i in range(kbest.shape[1]):
    # Logistic Regression
    acc_log = Logistic(
        X_train,
        y_train,
        X_test,
        y_test
)
    acclog.append(acc_log)

    # SVM Linear
    acc_sl = svm_linear(
        X_train,
        y_train,
        X_test,
        y_test
)
    accsvml.append(acc_sl)

    # SVM RBF
    acc_NL = svm_NL(
        X_train,
        y_train,
        X_test,
        y_test
)

    accsvmnl.append(acc_NL)

    # Decision Tree
    acc_d = Decision(
        X_train,
        y_train,
        X_test,
        y_test
)
    accdes.append(acc_d)

    # Random Forest
    acc_r = random(
        X_train,
        y_train,
        X_test,
        y_test
)
    accrf.append(acc_r)

# ==============================================================
# 17. CREATE FINAL RESULT
# ==============================================================
result = selectk_classification(
    acclog,
    accsvml,
    accsvmnl,
    accdes,
    accrf
)
# ==============================================================
# 18. DISPLAY RESULT
# ==============================================================
result
```

Out[47]:

| | Logistic | SVMl | SVMnl | Decision | Random |
|---|---|---|---|---|---|
| **ChiSquare** | 0.83 | 0.8 | 0.9 | 0.86 | 0.88 |

In [ ]: